UNIVERSITI MALAYA

REFINED QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

UMHackathon 2026

umhackathon@um.edu.my

Table of Content

Document Control **3**
FINAL ROUND (Execution, RCA & Hard Evidence) **3**
1. Test Execution & Evidence (Manual) 3
2. Automated Test Coverage Evidence 4
3. Defect Log & Fix Traceability (Root Cause Analysis) 5
4. Known Issues & Deferred Technical Debt 5
5. Live Demo / UAT Risks 6
6. QA Recommendation & Sign-Off 6

Document Control

Field	Detail
System Under Test (SUT)	*UMHackathon 2026 - Team Frogman*
Team Repo URL	*https://github.com/shaunn27/bizassist-ai-hub.git*
Project Board URL	*https://sharing.clickup.com/90182653398/b/h/6-901817666516-2/10f2bce3e9e3833*
Live Deployment URL	*https://bizassist-ai-hub-navy.vercel.app/*

Objective:

The primary objective of this refined QATD is to ensure that **BizAssist AI Hub produces accurate, reliable, and hallucination-free outputs across all AI-assisted workflows**, including order extraction, meeting scheduling, and the newly introduced **AI-powered invoice generation for SMEs**.

The system is designed to **correctly interpret unstructured customer chat inputs**, transform them into structured business actions, and generate **complete and valid invoice documents without fabricating any data**. It must handle edge cases such as incomplete orders, ambiguous customer requests, and missing financial details, while maintaining system stability and clear error handling.

Additionally, this QATD ensures that all AI outputs, especially invoice-related data such as item details, quantities, and totals, are **traceable to the original conversation source**, aligned with SME operational needs and Malaysia's evolving **e-invoicing practices**.

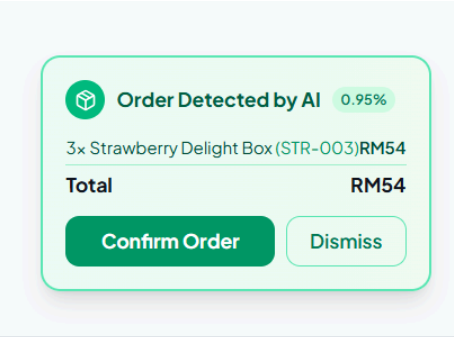
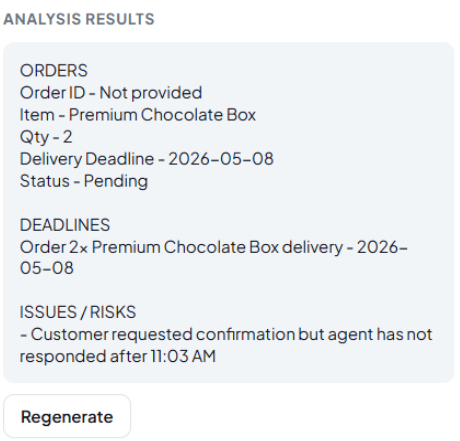
All execution results, defects, and AI behavior observations are documented with evidence, ensuring full transparency, reliability, and readiness for live demo and user acceptance testing.

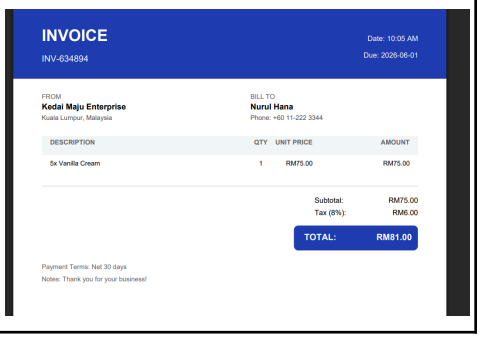
FINAL ROUND (Execution, RCA & Hard Evidence)

No evidence = No points.

1. Test Execution & Evidence (Manual)

*Execute the tests drafted in **Part 1**. You must provide a link to visual proof for at least 3 critical tests.*

Test Case ID	Test Type & Mapped Feature	Test Description	Expected Result	Actual Result	Status	Proof of Execution (Required for P0/Critical tests)
TC-01	***Happy Case (End-to-End Flow):*** *Chat → Order → Invoice Generation)*	*Verify AI extracts order from chat* *Then, Agent confirms* *Only then System (AI) will generate invoices* *1. Open Chat* *2. Click AI Help* *3. Agent confirm order* *4. Click "Generate Invoice"*	*Invoice generated with correct items, price, total, customer details. Stock also updated in database*	*Invoice generated successfully, values matched extracted order. Stock deducted from database*	✅ *Passed*	
TC-02	***Specific Case (Negative):*** *Edge Case, where customer's orders without full / correct required info*	*Verify that the system we built blocks invoice generation to avoid confusion / wrong info to customer or user*	*Invoice not generated OR shows "Missing Pricing Data" or any related error messages*	*System blocked invoice generation and displayed warning with details*	✅ *Passed*	

Test Case ID	Test Type & Mapped Feature	Test Description	Expected Result	Actual Result	Status	Proof of Execution (Required for P0/Critical tests)
TC-AI-01	***AI Output Validation:*** *Ready to send Invoice*	*We verify no hallucinated fields in invoice that is returned by Gemini*	*Only data from chat that is verified appears in invoice*	*All invoice fields are traceable to chat source with no error*	*Passed*	

2. Automated Testing Pipeline

2.1. Unit Testing (Core Logic & Functions)

Unit testing is primarily focused on Isolation of the individual components or functions to ensure that these functionalities work perfectly on their own without the need of any external dependencies. Mock your databases or APIs here.

- ***Test Runner Used:*** *Vitest*
- ***Targeted Components:***
  - `generateInvoiceData()`
  - `calculateTotalAmount()`
  - `validateInvoiceFields()`
- ***File(s) Covered:*** *[Link to the test file of your github repo]*

Unit Test ID	*Function/Component Tested*	*Test Scenario*	*Expected Outcome*	*Actual Result*	*Status*
UT-01	*calculateTotalAmount()*	*Multiple items*	*Correct sum and quantity*	*Correct sum and quantity*	✅ ***Pass***
UT-02	*generateInvoiceFata()*	*Missing address*	*Throws error*	*Error handled*	✅ ***Pass***

UT-03	*validateInvoiceFields()*	*Null fields*	*Reject invoice*	*Rejected*	✅ ***Pass***

Execution Proof*: [Screenshot of your terminal with the tests conducted or CI/CD proof of Github conducting the tests with results is also applicable]*

```
PS C:\Users\Asus\OneDrive\Documents\VS Code Project\UMH\bizassist-ai-hub>       npx vitest run tests/invoice.test.ts
    at eval (eval at runInlinedModule (file:///C:/Users/Asus/OneDrive/Documents/VS%20Code%20Project/UMH/bizassist-ai-hub/node_
modules/vite/dist/node/module-runner.js:913:9), <anonymous>:7:3)
    at ESModulesEvaluator.runInlinedModule (file:///C:/Users/Asus/OneDrive/Documents/VS%20Code%20Project/UMH/bizassist-ai-hub/
3:73)
 ✓ tests/invoice.test.ts (3 tests) 5ms
   ✓ Invoice System Core Logic (3)
     ✓ UT-01: calculateTotalAmount() - Multiple items should return correct sum and quantity 2ms
     ✓ UT-02: generateInvoiceData() - Missing address should throw error (handled) 1ms
     ✓ UT-03: validateInvoiceFields() - Null fields should reject invoice 0ms

 Test Files  1 passed (1)
      Tests  3 passed (3)
   Start at  07:26:43
   Duration  384ms (transform 61ms, setup 0ms, import 88ms, tests 5ms, environment 0ms)

close timed out after 10000ms
Tests closed successfully but something prevents Vite server from exiting
You can try to identify the cause by enabling "hanging-process" reporter. See https://vitest.dev/guide/reporters.html#hanging-
process-reporter
PS C:\Users\Asus\OneDrive\Documents\VS Code Project\UMH\bizassist-ai-hub>
PS C:\Users\Asus\OneDrive\Documents\VS Code Project\UMH\bizassist-ai-hub>
```

2.2. Integration & API Testing (System Modules)

This section validates the integration between core system modules, specifically the flow from AI processing to invoice generation and UI rendering. It ensures that data is correctly transmitted across components, stored reliably in the database, and accurately reflected in the user interface without errors or data inconsistency.

- ***Integration Testing (AI → Invoice → UI)***

- **Test Tool Used:** Postman (API testing), Manual UI Flow (Frontend validation)
- **Targeted Integrations:** Backend endpoints connecting to Supabase, Frontend component fetching data from backend API
- **File(s) Covered:** [e.g., Github repo showing Integration Tests or Postman Public Link]

Test ID	Integration / Point Tested	Test Scenario	Expected Outcome	Actual Result	Status
IT-01	Chat → AI → Invoice	This sends a valid order and verifies that the database record is created and correct status is updated	Invoice generated	Invoice successfully generated	✅ **Pass**
IT-02	AI → DB → Invoice	Stored order reused	Invoice correct	Success	✅ **Pass**

Execution Proof: [Screenshot of Postman Test Runner Results/ Terminal output of integration tests]

```
> postman run "BizAssist Integration Tests"

BizAssist API Tests
↳ POST /api/generate-invoice
  ✓ Status code is 200
  ✓ Response time is less than 5000ms
  ✓ Invoice data structured correctly

---------------------------------------------------------------------
1 iteration - 1 request - 3 test scripts

Executed in 1.23s
0 failed tests
```

3. Defect Log & Fix Traceability (Root Cause Analysis)

This section documents the key defects identified during testing, along with their root causes and corresponding fixes. Each issue includes clear reproduction steps, error logs, and a verified commit reference to ensure traceability and transparency. This demonstrates that all critical bugs were properly analysed, resolved, and validated within the codebase.

Bug ID	Bug Description	Steps to Reproduce	Console/Error Log Snippet	Root Cause Analysis (Why did it break?)	Fix Commit Hash / PR Link
DEF-01	*Invoice total calculated incorrectly*	*1. Confirm an order with multiple items.* *2. Click **Generate Invoice**.* *3. Compare invoice total with item quantity × unit price.*	Invoice total mismatch: expected calculated total but received wrong subtotal	*The calculation logic did not multiply quantity correctly before adding to subtotal.*	*Invoice-I…*
DEF-02	*Invoice generated with missing fields*	*1. Open a chat with incomplete customer details.* *2. Confirm order.* *3. Click **Generate Invoice**.*	ValidationError: missing required invoice fields: customerName / address / itemPrice	*The invoice module allowed generation before checking all required fields. Missing validation caused incomplete invoice output.*	*Invoice-I…*
DEF-03	*Duplicate invoices created*	*1. Confirm an order.* *2. Click **Generate Invoice** multiple times quickly.* *3. Check invoice list/history.*	Duplicate invoice ID detected	*The button remained active during invoice generation, allowing repeated requests before the first process completed.*	*Duplicat…*

4. Known Issues & Deferred Technical Debt

This section outlines minor issues and technical limitations identified during development that were not resolved due to time or scope constraints. These items do not affect core functionality but are documented to demonstrate transparency, proper risk awareness, and planned future improvements.

Bug ID	Description & Impact (Bug or Technical Debt)	Reason for Deferral	GitHub Issue Link
DEF-04	*Bulk Invoice Export not supported (Only single PDF export works currently)*	*Time constraint*	github.com/shaunn27/bizassist-ai-hub/issues/4
DEF-05	*Tax calculation not included (only mock percentage)*	*Future enhancement*	*github.com/shaunn27/bizassist-ai-hub/issues/5* *Invoice-INV-846807.pdf*
DEF-06	*No official LHDN e-Invoice format yet*	*Pending compliance*	*github.com/shaunn27/bizassist-ai-hub/issues/6* *Invoice-INV-846807.pdf*

5. Live Demo / UAT Risks

- Risk 1: Incomplete Chat → Incomplete Invoice
If the customer message lacks key details (e.g., price, address, quantity), the AI may not generate a complete invoice. Judges are advised to use clear and complete order inputs during testing, or expect the system to prompt for missing information instead of generating an invoice.

- Risk 2: AI Processing Delay (1–2 seconds)
AI analysis and invoice generation may take around **1–2 seconds** depending on input length. Please avoid clicking the “AI Help” or “Generate Invoice” button multiple times during processing, as the system is designed to handle one request at a time.

- Risk 3: Prototype Limitation (Not Connected to Official e-Invoice API)
The current invoice generation is a **simulation for SME workflows** and is not yet integrated with Malaysia’s official LHDN e-Invoice system. Generated invoices are structurally correct but not legally submitted or validated through government APIs.

- Risk 4: Browser Optimization
This prototype is optimized for **desktop usage on Google Chrome**. Other browsers or mobile devices may experience minor UI inconsistencies or layout issues.